

# DataPact: Know When Your Data Breaks, and Why

Hriday Agarwal\*, Ashok Chacko\*, Tejas Jaggi\*, Connor Slattery\*

University of Illinois Urbana-Champaign

\*All team members are considered equal contributors.

**Abstract**—Many teams depend on datasets that arrive repeatedly in the same form, such as a monthly sales export or a term-by-term enrollment file. The people who use these files often have clear expectations about their contents, but those expectations are usually undocumented and rarely checked. When a new file quietly departs from them, the problem may not be discovered until it has already affected a report or analysis. This paper describes DataPact, a web application that lets a team define what a recurring dataset is supposed to contain, check each new version against those expectations, and keep a record of validation results over time. Because validation is performed using explicit rules, the application can be tested against known inputs and expected outcomes. This work is submitted by Team DataPact, Team Number 15.

## I. IDEATION

Our team is building a single application progressively across the semester, so we wanted an idea that could begin with a clear, testable foundation and grow in complexity over time. Before comparing specific proposals, we agreed on several criteria for selecting the project. We wanted a problem that could be explained clearly to someone unfamiliar with it, realistic data that could be used for demonstrations, and a core whose results could be evaluated against known inputs and expected outcomes. We also wanted the core application to be useful without artificial intelligence, leaving room for AI to provide additional assistance later rather than making it responsible for the primary result. Finally, the idea needed enough depth to support the progressive requirements of the course, including the later public interface and AI-related assignments.

We considered three ideas seriously.

The first was QueuePilot, an incident management tool for engineering teams. The concept would organize application incidents and help teams summarize and manage related failures. We liked the idea and its technical depth, but it was broader and more complicated to build well within the semester. We were concerned that too much of the project would be spent on interpreting and organizing incident information rather than developing a focused application that we could complete and demonstrate effectively.

The second was RubricLens, a tool for examining grading consistency across teaching assistants. We saw value in the problem and thought it could support an interesting data model. However, the project would depend on student grades and written feedback, which are sensitive information. This would make it difficult to demonstrate the public data interface required later in the course using real data. Relying mainly on

fabricated grading records would also make the demonstration less convincing.

We selected DataPact because it provided a clearer balance between usefulness, feasibility, and long-term project potential. Public datasets can be used as realistic examples, and deliberately corrupted copies can create known validation failures without inventing an entire dataset. Because the validation rules are explicit, the application can be tested against known inputs and expected outcomes. The core is also useful without artificial intelligence, giving the team a dependable foundation before introducing AI assistance. Finally, the project can grow naturally as later course assignments introduce APIs, authentication, deployment, and advanced AI capabilities.

We did not conduct user interviews, surveys, or formal market research, and we did not approach a registered student organization for this project. The alternatives were generated and evaluated through team discussion, supported by the AI tools noted in the acknowledgement.

## II. INTRODUCTION

A large amount of routine analytical work depends on datasets that are produced repeatedly. A university office may export enrollment information each term, a finance team may receive a sales file each month, or a research group may collect measurements on a fixed cycle. Each new file is expected to follow certain assumptions about its structure and contents.

These expectations are often real but rarely written down. They may exist only in the memory of the person who prepares the file or the person who uses it: a column should always be present, a field should never be empty, or a value should belong to a particular set of categories. When these expectations are informal, a new file can violate them without anyone noticing. The file may continue into a dashboard, analysis, or other downstream process before the problem becomes visible.

We use the term *data contract* for a written set of expectations describing what a dataset should contain and what rules its data should satisfy. DataPact is a web application that allows users to define these expectations and check new files against them.

## III. MOTIVATION

The cost of an unnoticed data problem is not only the effort required to check files manually. It is the time between the moment a problem enters a dataset and the moment someone notices it. During that time, affected values may spread into dashboards, analyses, models, and decisions. Tracing the

problem backward can then require substantially more effort than identifying it when the file was first received.

Writing expectations down also turns private assumptions into a shared reference. A team can agree on what a column means, which values are valid, and when a value may be missing before those disagreements affect downstream work. Software development addresses a similar problem through automated testing, but recurring data is often still checked manually or inconsistently.

#### IV. PROPOSED FEATURES

DataPact provides four core capabilities.

**Dataset management.** A user registers a recurring dataset and records basic information about it so that its files and validation results remain organized in one place.

**Data contract management.** For each dataset, the user defines the expectations its files must satisfy. These may specify required columns, expected data types, permitted values, acceptable ranges or formats, and whether values may be missing.

**Data validation.** When a new file is uploaded, the application checks it against the selected expectations. The results identify which rules failed, how many rows were affected, and which values caused each failure.

**Validation history and quality trends.** Each validation run is stored so that users can review previous results, identify recurring problems, and see whether the quality of a dataset is improving or deteriorating over time.

#### V. APPLICATION FUNCTIONALITY

The workflow is straightforward. An analyst registers a recurring dataset and defines its expectations. When the next version of that file arrives, the analyst uploads it and selects the expectations to check against. DataPact applies each rule and stores the result as a validation run. If the file passes, the successful run becomes part of the dataset's history. If the file fails, the application identifies each broken rule, the number of affected rows, and examples of the values responsible, allowing the user to investigate the actual data rather than seeing only a failure count.

An important part of the idea is that a failed check does not always mean the incoming data is wrong. Sometimes, the expectation itself has become out of date. For example, a dataset may historically contain only the status values `ACTIVE` and `INACTIVE`, but the organization may later introduce a legitimate `AUDIT` status. The new value would violate the existing expectation even though the underlying data is correct. DataPact is intended to help the user distinguish between data that needs to be fixed and a legitimate change that requires the dataset's expectations to be updated.

This makes DataPact more than a tool for checking an individual file. The expectations and validation history remain associated with the recurring dataset, allowing a team to maintain a clearer record of what the data is expected to contain and how its quality changes over time.

#### VI. ROLE OF AI

DataPact is designed so that artificial intelligence assists people using the application without being responsible for validation correctness. Whether a file passes or fails is determined by explicit rules, and the same file checked against the same expectations should produce the same result.

Later in the project, AI can help in several areas. It could propose an initial set of rules after examining a sample dataset, explain in plain language why a validation check failed, or summarize problems that recur across a dataset's history. These outputs will remain advisory. A person would review and approve any suggested rule before it takes effect, and AI would not make the core pass/fail decision.

#### VII. SCOPE

The initial version of DataPact focuses on datasets supplied as comma-separated value (CSV) files. Validation occurs when a user uploads a file rather than through scheduled ingestion. Automated collection from external systems, alerting and notifications, integration with existing data pipelines, and statistical anomaly detection are outside the initial scope of the project.

#### ACKNOWLEDGEMENT

The team used OpenAI ChatGPT and Anthropic Claude as brainstorming and critique tools during project ideation and while drafting this document. These tools were used to generate and evaluate candidate concepts. The team independently discussed the alternatives and made the final design and project selection decisions described in this document.

#### REFERENCES

- [1] dbt Labs, "Data tests," *dbt Documentation*. [Online]. Available: <https://docs.getdbt.com/docs/build/data-tests>
- [2] Great Expectations, "Great Expectations Documentation." [Online]. Available: <https://docs.greatexpectations.io/>